

4.3.3 compare-modes

```
python ir_system/skills_ir_system.py compare-modes --eval-set core_relevance.json
```

这是消融实验的核心命令：在同一个评测集上，分别以 tfidf、bm25、hybrid 三种模式各跑一遍完整的评测流程，横向比较聚合指标（Hit@K、MRR@K、Recall@K、Precision@K），并标注每个评测集上哪种模式胜出。如果 --eval-set 传多个评测集（例如同时比较 core_relevance.json 和 multilingual.json），报告会按评测集拆开，方便观察不同语言分布下各模式的优劣变化。

与 evaluate 的区别在于：evaluate 只跑一种模式、看绝对分数；compare-modes 同时跑三种模式、看谁在什么条件下更优。对于回答“关掉 BM25 会怎样”“TF-IDF 单独用够不够”这类问题，这条命令直接给出量化答案。结果写入 ir_system/runtime/comparison_report.json，同时生成一份同名的 Markdown 文件供阅读。

4.3.4 search

```
python ir_system/skills_ir_system.py search "Qdrant_vector_search" --top-k 5
```

端到端查询命令。流程为：加载索引 → 对查询文本分词并扩展 → 计算混合得分（TF-IDF 余弦 + BM25 + 动态权重融合）→ 施加后置 boost（标题匹配、token 覆盖率、热度修正等）→ 按标准化标题去重（每个 skill name 只保留最高分的一条）→ 返回 top-K 排序结果，终端打印 skill_name 和得分，同时将完整结果（含查询文本、模式、结果列表和索引摘要）写入 ir_system/runtime/search_results.json。

这条命令不涉及评测集和 Ground Truth——不计算 Precision/Recall，纯粹用于人工感知检索质量：搜一句话，看返回的前几条 skill 是否相关、排序是否合理。在调参阶段频繁使用，用于快速验证一个改动（例如调整 alpha 分段阈值或修改扩展词典）对排序的实际影响。

5 实验结果与对比分析

5.1 三种检索模式对比

表 3 汇总了 core_relevance.json 上的主要结果。Hit@5 三种模式都是 1.0，说明相关结果至少都能进前五；真正拉开差别的是排序位置和前五名里相关结果的占比。

表 3: 三种检索模式在 core_relevance.json 上的表现对比

模式	P@5	nDCG@5	MRR@5	Hit@5
TF-IDF	0.2762	0.8962	0.9107	1.0000
BM25	0.2714	0.8671	0.8996	1.0000
Hybrid	0.2810	0.8988	0.9087	1.0000

这个结果挺符合直觉。纯 TF-IDF 的 MRR@5 最高，说明在把最相关结果顶到第 1 位这件事上，向量空间模型还是主力。BM25 单独跑时不算差，但在这批数据上没有超过 TF-IDF。混合检索的作用更像“保守补分”：它没有把 MRR 再抬高，但把 P@5 和 nDCG@5 稍微往前推了一点，前五名整体更整齐。

5.2 BM25 权重扫描

为了看清楚 α 该压在什么区间，我把 BM25 权重固定后重新扫了一遍。结果没有单独放图，但结论很稳定： $\alpha = 0.02 \sim 0.05$ 一段最稳，继续加大 BM25 占比后， nDCG@5 和 MRR@5 会缓慢下滑，纯 BM25 的退化最明显。

这个扫描结果说明了两件事。第一，BM25 确实能帮一点忙，但只适合少量掺进去。第二，当前代码里那套“中文/短查询权重更低，长英文查询再稍微提高”的策略不是拍脑袋定的，它正好落在较稳的区间附近。

5.3 中文、英文与 mixed 查询差异

表 4 是 hybrid 模式按查询类型拆开的结果。英文查询的平均 α 为 0.0942，中文和 mixed 查询都压到 0.02。

表 4: Hybrid 模式按查询类型拆开的表现与平均 BM25 权重

查询类型	数量	平均 α	P@5	nDCG@5	MRR@5
English	24	0.0942	0.2750	0.9174	0.9410
Chinese	6	0.0200	0.2667	0.9385	0.9167
Mixed	12	0.0200	0.3000	0.8419	0.8403

这里最有意思的是中文查询。虽然它们的 BM25 权重最低， nDCG@5 反而不差。这和数据分布有关：文档主体几乎全英文，中文查询能命中，主要靠扩展词典把“前端”“结构化数据”“向量检索”映到对应英文术语上。这一步成功以后，TF-IDF 的向量相似度已经能把主相关结果推上来；BM25 再往上加，只会让零散中文 token 和局部词面匹配掺进更多噪声。

我又用 `multilingual.json` 做了交叉检查。那套评测里中文查询只有 4 条，纯 TF-IDF 和 hybrid 的 nDCG@5 都是 0.6577，BM25 则掉到 0.5886。这和式 (8) 的方向是一致的：中文部分先别让 BM25 说太多话。

5.4 失败案例

虽然三种模式的 Hit@5 都是 100%，但还有几个典型问题值得记一下。

- `copywriting for marketing`: 相关 skill 在前五里, 但 `marketing-skills-collection` 和 `marketing-automation` 这类泛化程度高的结果会抢到更前面, 说明领域词过宽时, 主题相关性和“任务是否对题”还没有完全拆开。
- `browser automation`: 相关结果有多个, `actionbook` 常常排不进更靠前的位置, 说明标题和描述里直说“`browser automation`”的 skill 更占便宜, 隐式相关项吃亏。
- `SEO audit:seo-audit` 往往能到第 1, 但另一个期望结果 `domain-authority-auditor` 不够靠前, 暴露出多相关文档场景下的覆盖问题。
- `SQL queries: supabase-postgres-best-practices、database-optimizer` 这类近邻文档有时会把 `sql-queries` 挤到第 3 或第 4, 说明“强相关概念邻居”对纯词法模型还是个老问题。

6 总结与展望

这套 IR 系统跑下来, 最大的感受是“简单模型不等于简单处理”。真正有用的不是单独的 TF-IDF 或 BM25, 而是整套围绕数据特征做的小修补: 字段加权、查询扩展、动态 α 、标题和热度 boost。当前数据上, TF-IDF 还是主骨架; BM25 更适合做轻量补分, 权重大了反而坏事。

局限也很清楚。第一, 文档侧英文占绝对多数, 中文检索基本靠手工扩展词典兜着。第二, 评测脚本里还没有原生 `nDCG`, 想看排序细节得另外补算。第三, 规则 boost 现在是手调常数, 还没有学到更细的排序偏好。后面如果继续做, 我会优先补三件事: 扩中文扩展词典、把 `nDCG` 正式纳入 CLI 评测、再加一层轻量 reranker, 把“概念相关但不是答案”的结果往后压一点。